SECV3213 FUNDAMENTAL OF IMAGE PROCESSING

SESSION 2025/2026 SEMESTER 1

SECTION 2

PROJECT REPORT

iPixelo: A Mobile-Inspired Image Editing and Enhancement Application

LECTURER'S NAME: DR. MD. SAH BIN HJ. SALAM

GROUP NAME: INNOVATORS

NO.	GROUP MEMBERS	MATRIC NO
1	CHUAH CHUN YI	A23CS0070
2	CHONG JUN HONG	A23CS0066
3	TAI YI TIAN	A23CS0272

TABLE OF CONTENTS

1.0 Introduction.....1
2.0 Application Interface & Feature Explanation..... 2
2.1 Home Screen & Upload Image.....2
2.2 Crop Image.....3
2.3 Rotate Image.....4
2.4 Flip Image.....5
2.5 Adjust Image Brightness.....6
2.6 Adjust Image Blur.....7
2.7 Adjust Image Noise Reduction.....8
2.8 Adjust Image Sharpness.....9
2.9 Adjust Image Saturation.....10
2.10 Apply Image Filter.....11
2.11 Insert Text for Image.....12
2.12 Save Image.....13
3.0 Process Flow Diagram..... 14
3.1 Overall Application Process Flow.....14
3.2 Image Cropping Process.....15
3.3 Image Rotation Process.....15
3.4 Image Flipping Process.....16
3.5 Brightness Adjustment Process.....16
3.6 Image Blurring Process.....17
3.7 Noise Reduction Process.....17
3.8 Image Sharpening Process.....18
3.9 Saturation Adjustment Process.....19
3.10 Visual Filtering Process.....20
3.11 Text Insertion Process.....21
4.0 Explanation of Process Flow.....22
4.1 Overall Application Process Flow.....22
4.2 Image Cropping Process.....23
4.3 Image Rotation Process.....24
4.4 Image Flipping Process.....25
4.5 Brightness Adjustment Process.....26
4.6 Image Blurring Process.....27
4.7 Noise Reduction Process.....28
4.8 Image Sharpening Process.....29
4.9 Saturation Adjustment Process.....30

4.10 Visual Filtering Process..... 31
4.11 Text Insertion Process..... 32
5.0 Results & Discussion.....33
5.1 Crop Image.....33
5.2 Rotate Image..... 34
5.3 Flip Image..... 35
5.4 Adjust Image Brightness.....36
5.5 Adjust Image Blur.....37
5.6 Adjust Image Noise Reduction..... 38
5.7 Adjust Image Sharpness.....39
5.8 Adjust Image Saturation..... 40
5.9 Apply Image Filter.....41
5.10 Insert Text for Image.....42
5.11 ROI Based Adjustment..... 43
6.0 Workload Distribution..... 44
7.0 Conclusion..... 45

1.0 Introduction

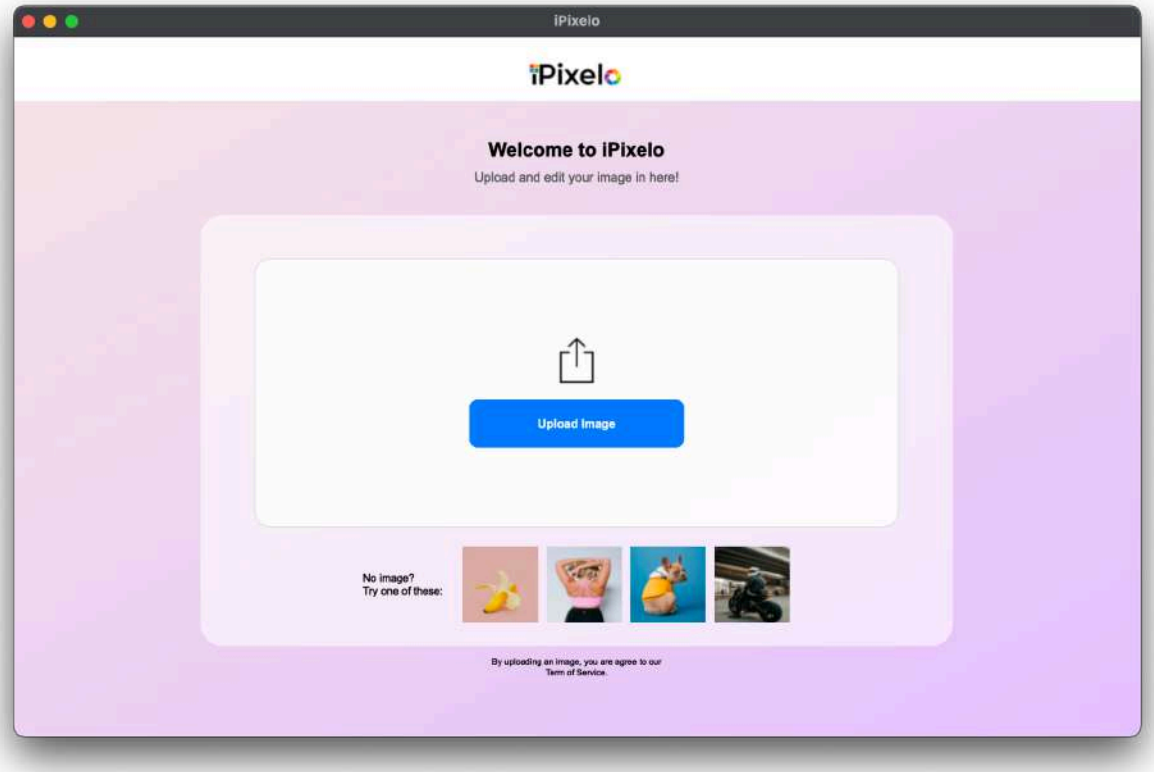
Digital images are widely used in various application domains such as social media, photography, entertainment, healthcare, industrial automation and surveillance. As a result, there is a greater demand for more effective image processing techniques that manipulate digital images to enhance image quality, restoring image details and highlighting important features. Fundamental image processing operations include image enhancement through brightness and contrast adjustment, image filtering through noise reduction or sharpness adjustment and geometric transformations such as rotation and cropping which are commonly used to correct image imperfections and support visual analysis.

In this project, a mobile-inspired image editing and enhancement application which is called iPixelo is developed using Python, the OpenCV library and the PyQt6 based graphical user interface (GUI).This application is designed to showcase the practical application of image processing techniques through an interactive system. By integrating OpenCV's image processing capabilities with PyQt6's GUI framework, iPixelo provides users with an intuitive platform for performing real-time image editing and enhancement operations. iPixelo allows users to upload desired images and perform a wide range of image processing operations, including brightness adjustment for image brightening or darkening, blurring, sharpening, noise reduction, saturation adjustment and ROI based processing for targeted editing operations. Besides, it enables users to perform other common editing operations such as cropping, flipping, rotating, text insertion and visual filtering effects. After completing editing tasks, iPixelo enables users to preview the processed results instantly and save the final edited image to their local storage.

The primary objective of this project is to integrate and apply fundamental image processing concepts and techniques learned in this course into a functional interactive application. The project focuses on the implementation of essential image enhancement, filtering, and geometric transformation operations, including brightness and contrast adjustment, noise reduction, blurring, sharpening, rotation, cropping, and ROI-based processing. Through the use of the PyQt6 GUI with interactive parameter controls and real-time visual feedback, the application enables users to easily observe, evaluate, and understand the effects and effectiveness of each implemented image processing operation.

2.0 Application Interface & Feature Explanation

2.1 Home Screen & Upload Image



Purpose:

To introduce the iPixelo application by providing users with a clear starting point upon launching the application and allowing users to upload an image they want to edit.

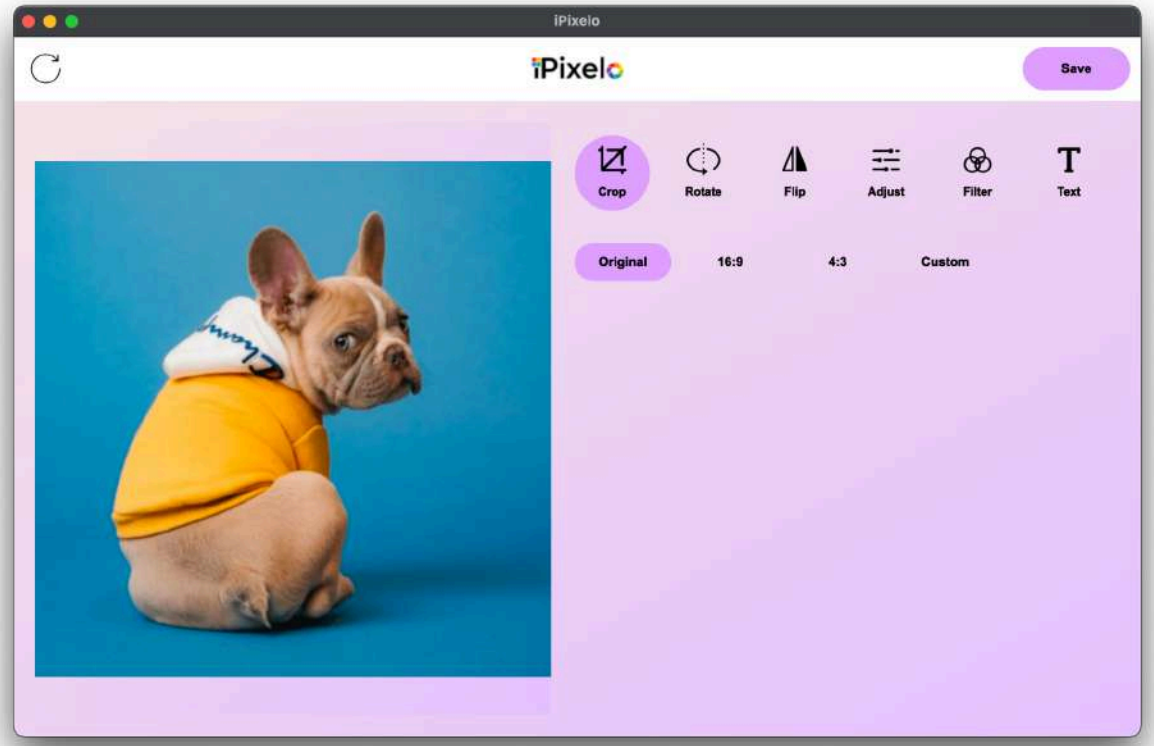
User Interaction:

Users can click the “Upload Image” button to choose an image file (e.g., JPG or PNG) or select any sample images provided.

Output:

After an image is selected, the interface switches from the home screen to the image editing interface, where editing tools and menus become available.

2.2 Crop Image



Purpose:

Removes unwanted regions and adjusts image composition.

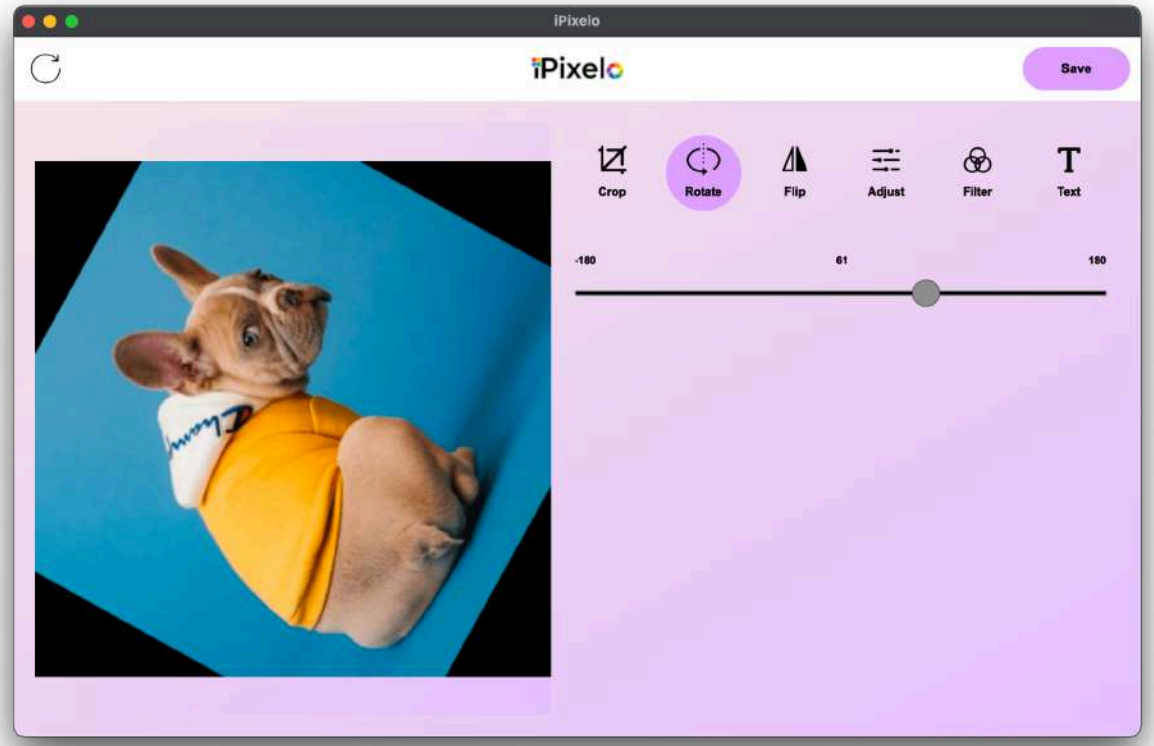
User Interaction:

Users can select predefined aspect ratios such as 16:9, 4:3 or a custom ROI to crop the image.

Output:

The image is cropped based on the selected region.

2.3 Rotate Image



Purpose:

Change the orientation of the image by rotating it around its center.

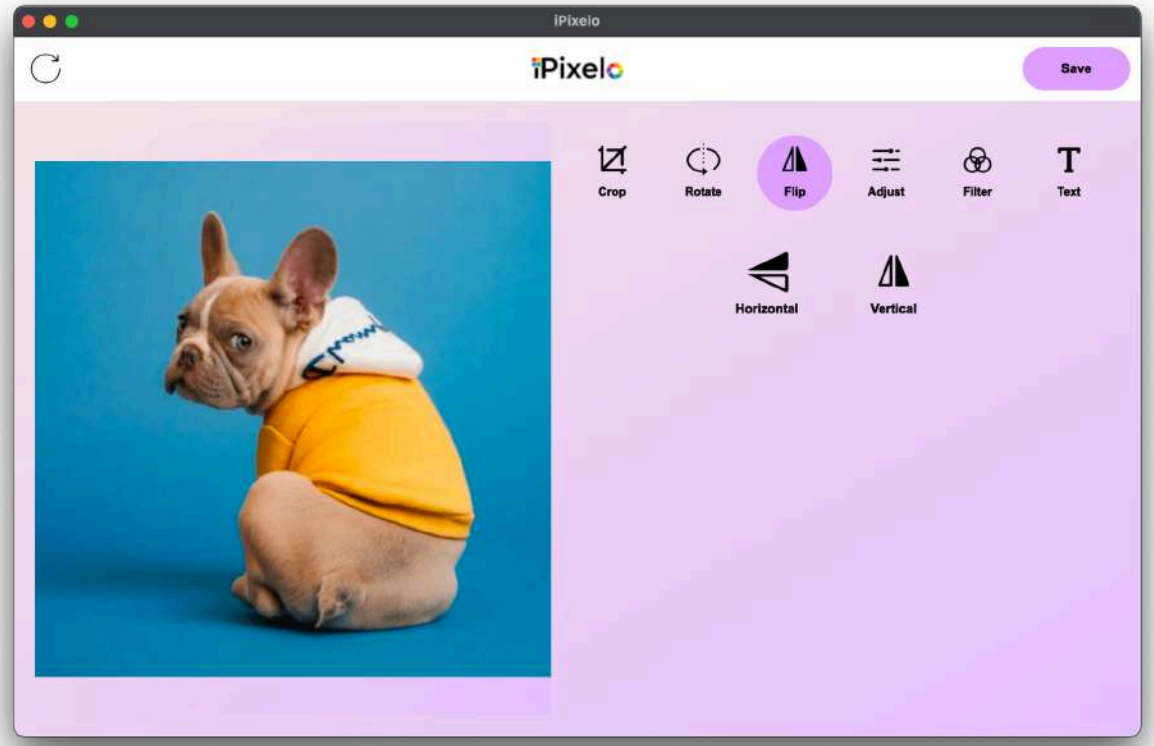
User Interaction:

The user selects the rotation operation and uses a slider to adjust the angle. Values less than 0 rotate the image clockwise, while values greater than 0 rotate it anticlockwise.

Output:

The image is rotated to the specific angle selected by the user.

2.4 Flip Image



Purpose:

Create a horizontal or vertical mirror of an image.

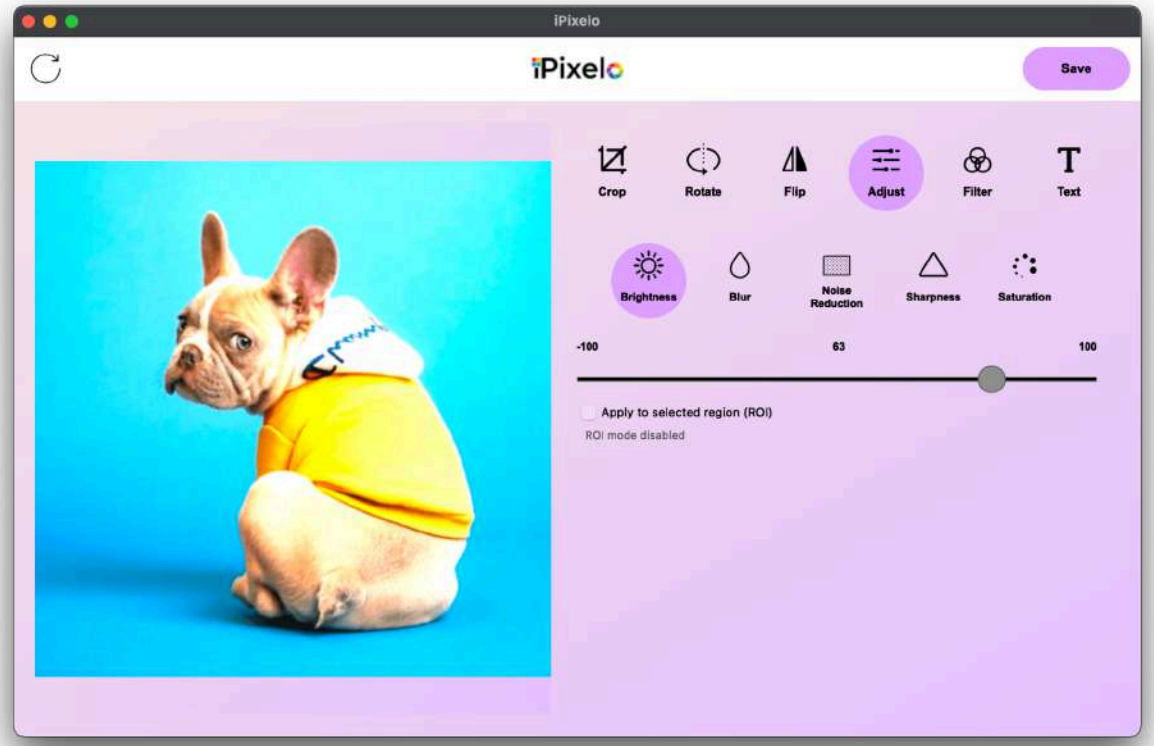
User Interaction:

The user chooses the flip operation and selects either "Horizontal" or "Vertical" to flip the image.

Output:

The image pixels are reversed along the selected axis which is horizontal or vertical.

2.5 Adjust Image Brightness



Purpose:

Modify the lightness or darkness of the image.

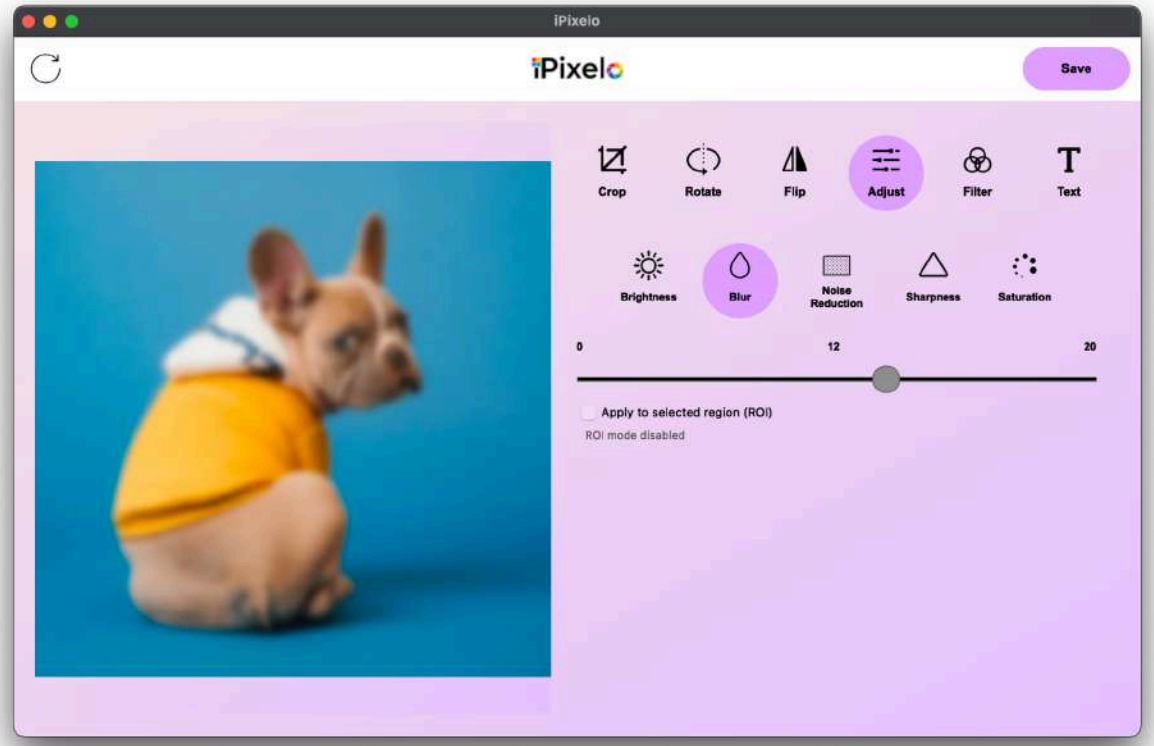
User Interaction:

The user selects the brightness operation inside the adjust and uses a slider to adjust the brightness. Values less than 0 darken the image, while values greater than 0 brighten it. Users can also click the "Apply to selected region (ROI)" to only apply the adjustment on the region selected by users.

Output:

The intensity of the pixel in the image increases or decreases based on the slider value.

2.6 Adjust Image Blur



Purpose:

Reduce the detail of the image.

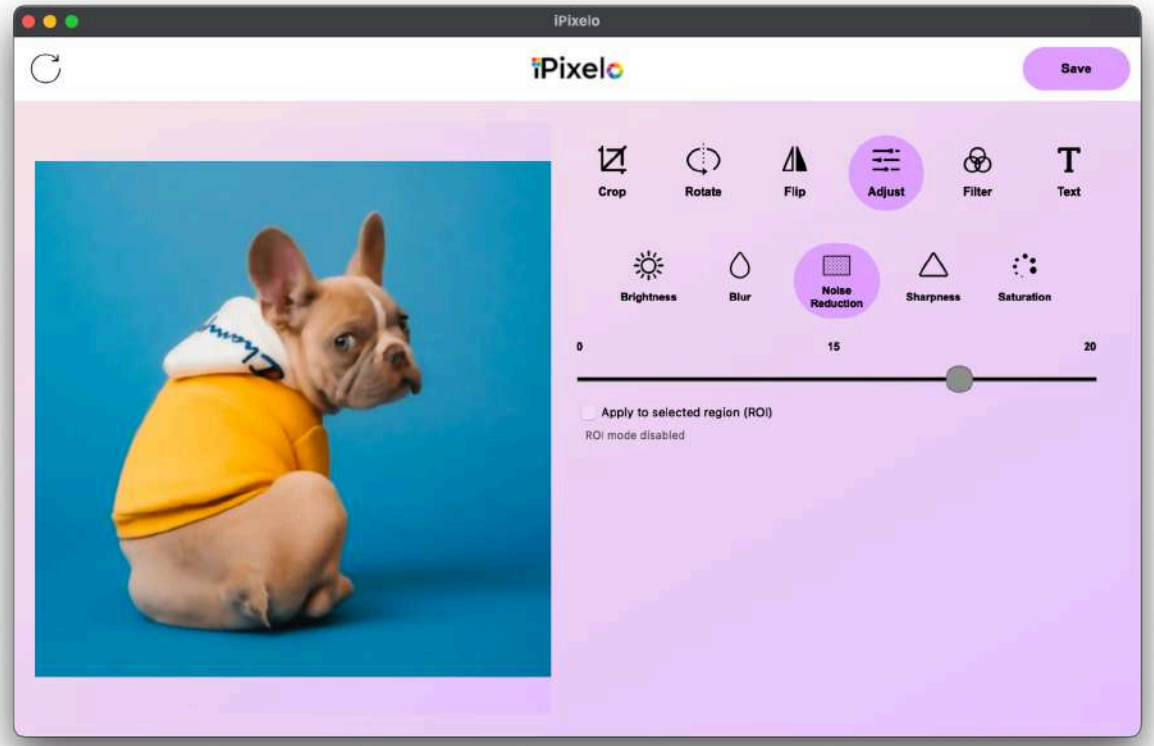
User Interaction:

The user selects the blur operation inside the adjust and uses a slider to adjust the level of blur effect. Users can also click the "Apply to selected region (ROI)" to only apply the adjustment on the region selected by users.

Output:

The selected area or whole image appears softer or out of focus depending on the chosen blur level.

2.7 Adjust Image Noise Reduction



Purpose:

Improve the image quality by removing the noise on the image.

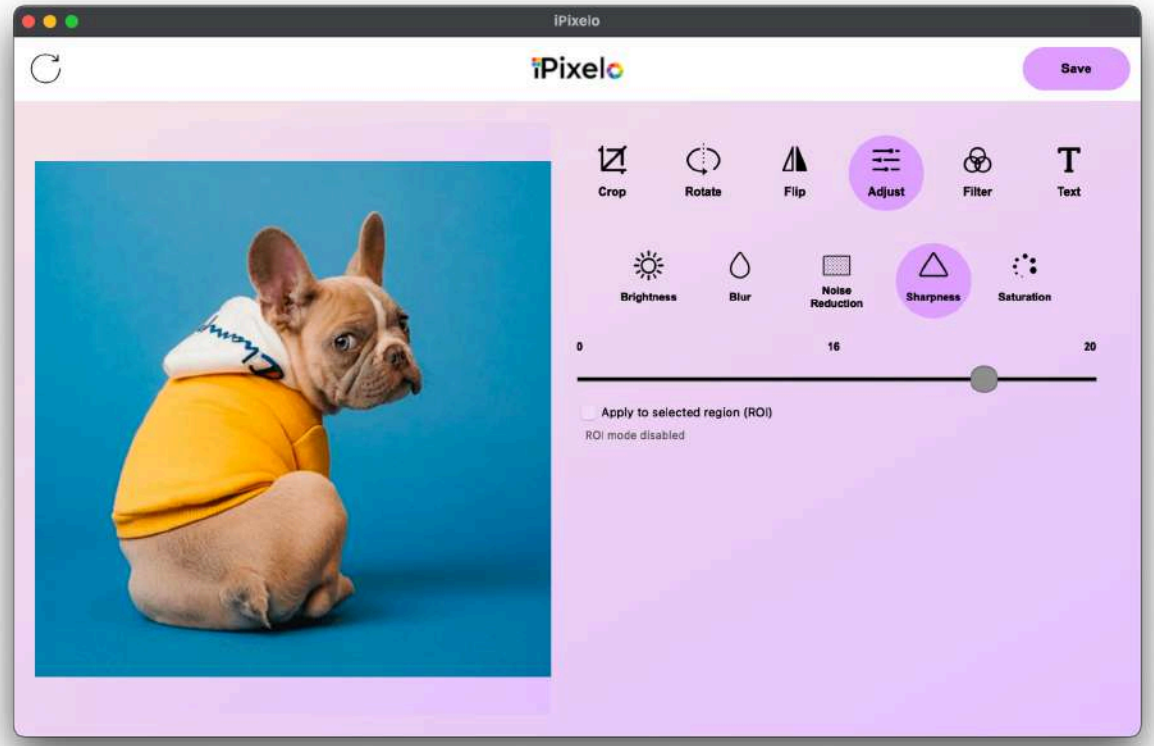
User Interaction:

The user selects the noise reduction operation inside the adjust and uses a slider to adjust the strength of the noise reduction filter. Users can also click the "Apply to selected region (ROI)" to only apply the adjustment on the region selected by users.

Output:

The image appears smoother with less noise.

2.8 Adjust Image Sharpness



Purpose:

Enhance the definition of edges in an image, making it appear crisper and more detailed.

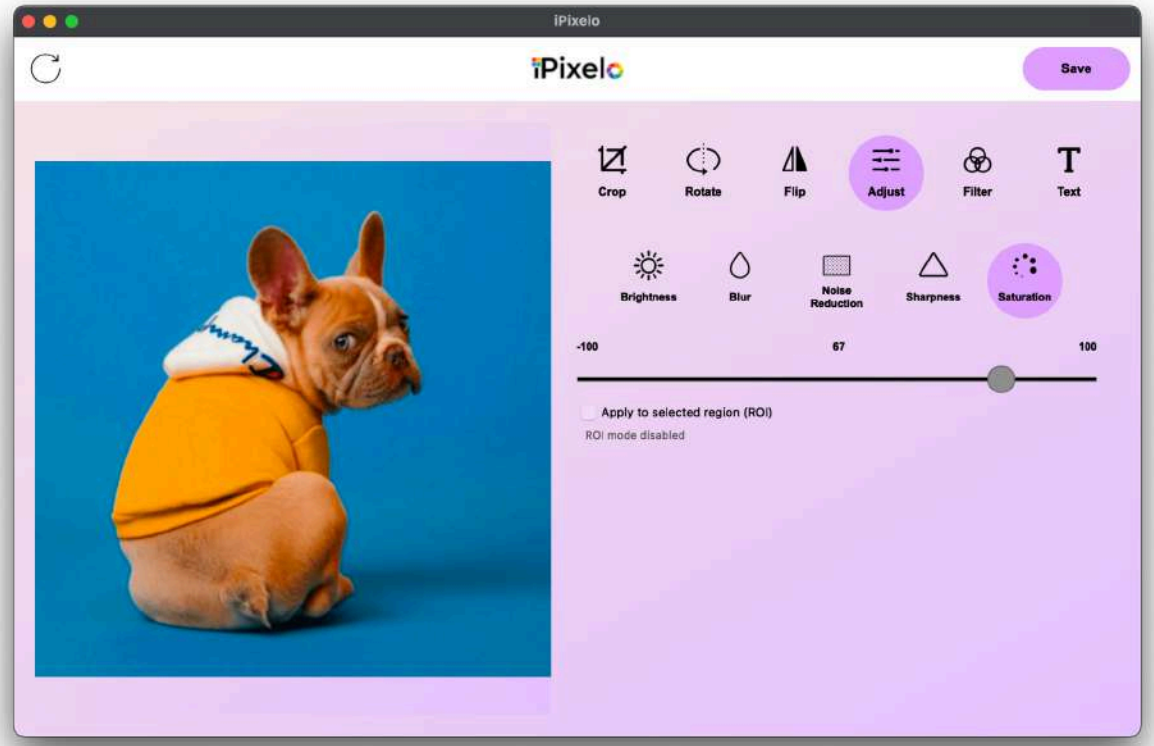
User Interaction:

The user selects the sharpness operation inside the adjust and uses a slider to adjust the sharpness level. Users can also click the “Apply to selected region (ROI)” to only apply the adjustment on the region selected by users.

Output:

The edges and details within the image become more distinct and pronounced.

2.9 Adjust Image Saturation



Purpose:

Increase or decrease the intensity of colors in the image.

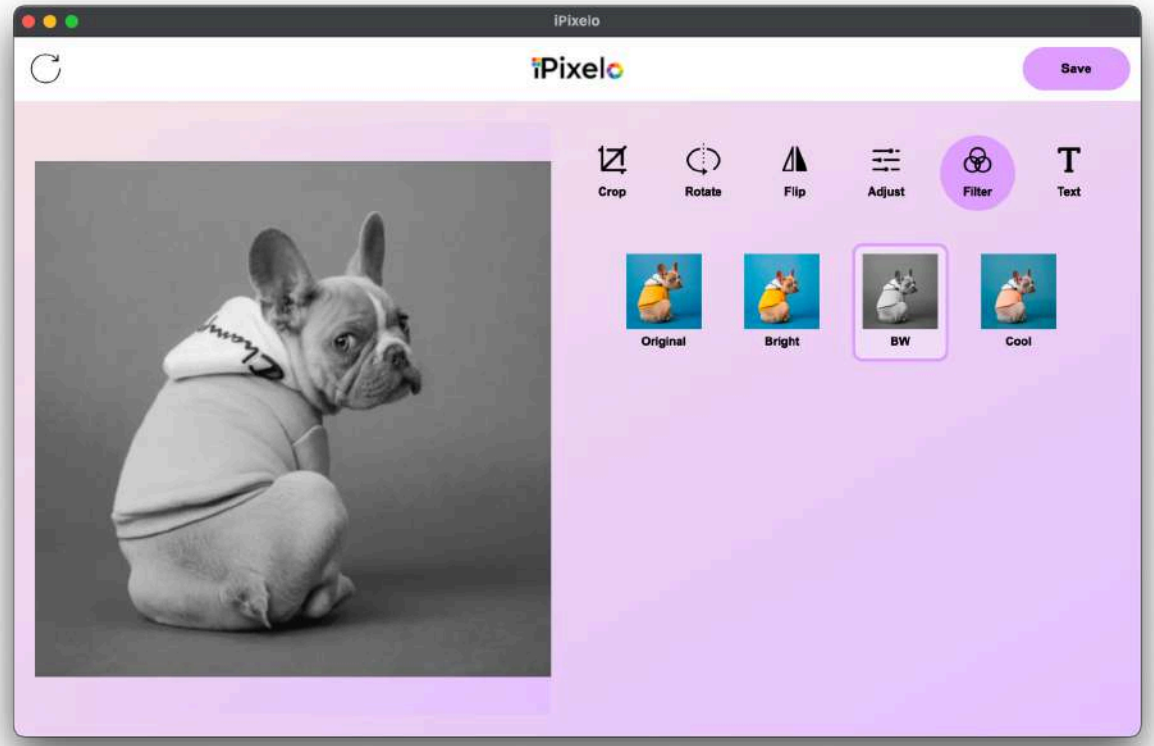
User Interaction:

The user selects the saturation operation inside the adjust and uses a slider to adjust the saturation level. Values < 0 desaturate the image which makes it look faded and values > 0 increase saturation. Users can also click the "Apply to selected region (ROI)" to only apply the adjustment on the region selected by users.

Output:

The colorfulness of the image is altered, ranging from grayscale to highly vivid colors.

2.10 Apply Image Filter



Purpose:

Apply different styles for the image.

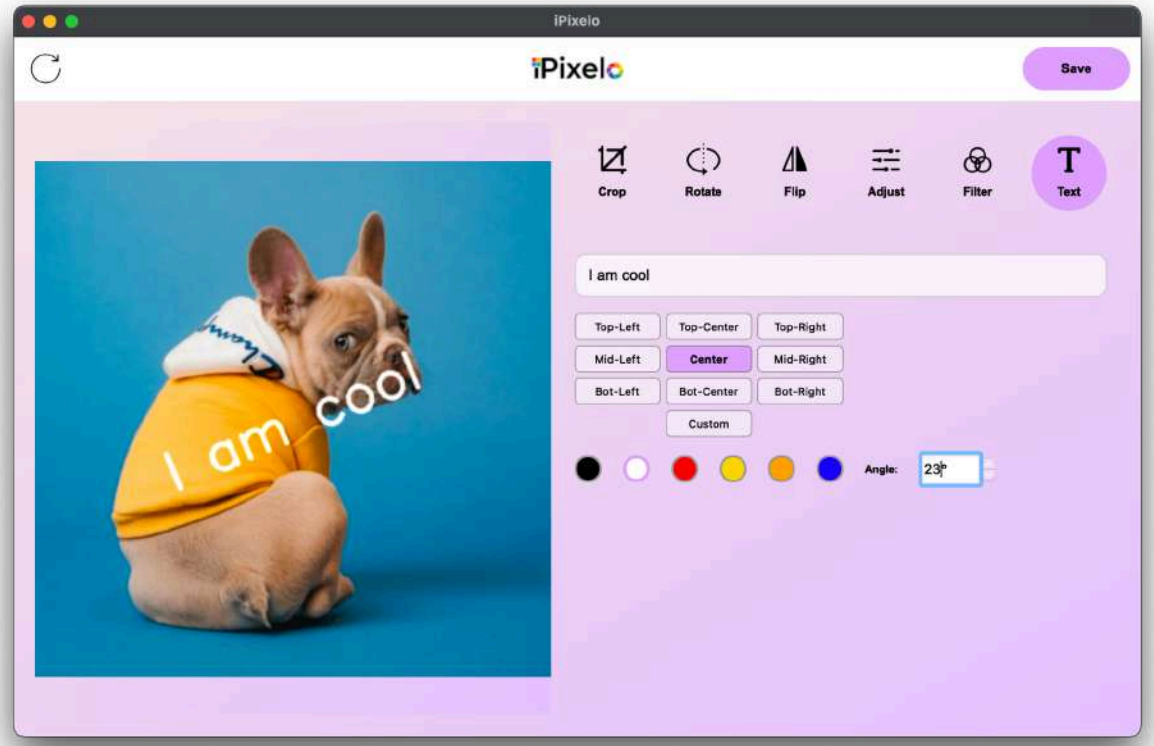
User Interaction:

The user selects the filter operation and chooses a specific filter style to apply for the image.

Output:

The colors and tones of the image will be transformed according to the selected filter preset.

2.11 Insert Text for Image



Purpose:

Place a text into the image to display.

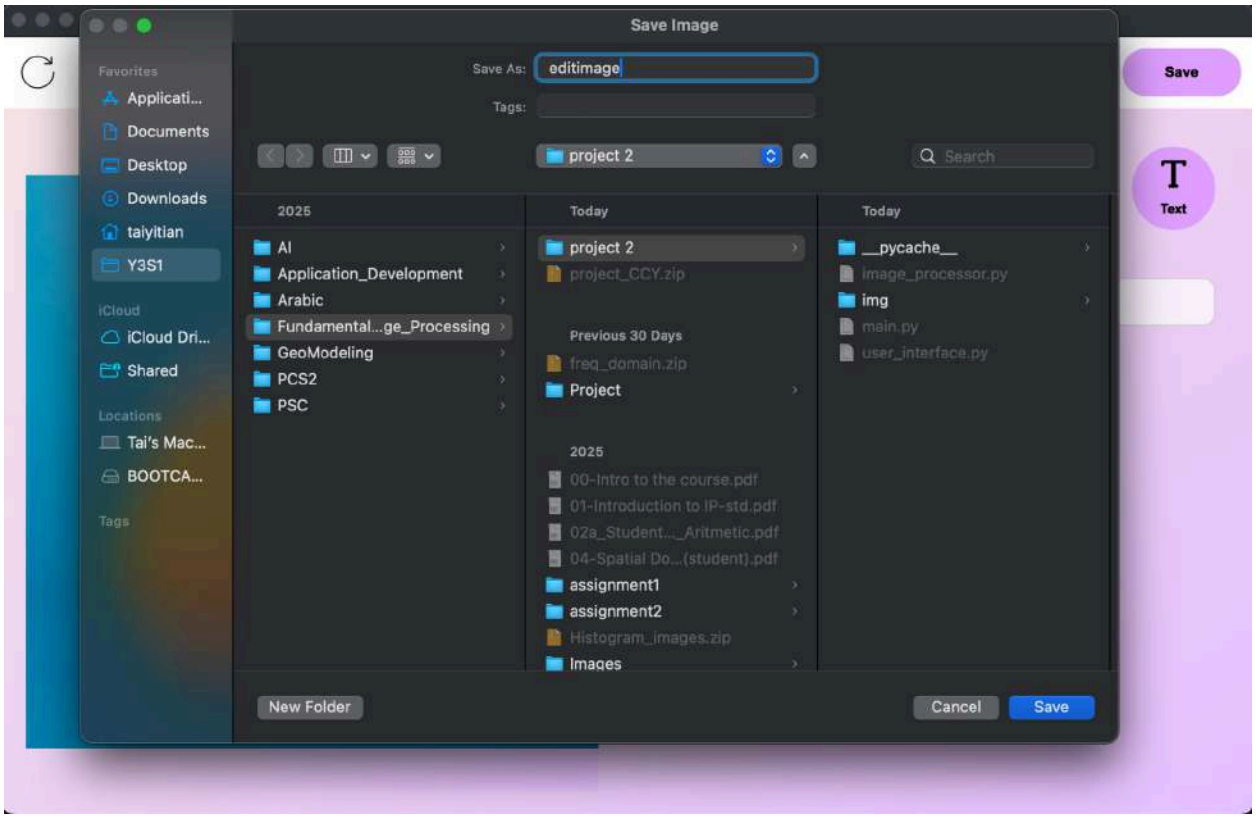
User Interaction:

The user selects the text operation and types the text they want. Users can choose the location of text in the picture such as center or custom to give the x and y coordinates. Users can also select the color and also the rotation angle for the text.

Output:

The text appears in the specified location with the color and rotation that have been set.

2.12 Save Image



Purpose:

To store the edited image into computer memory.

User Interaction:

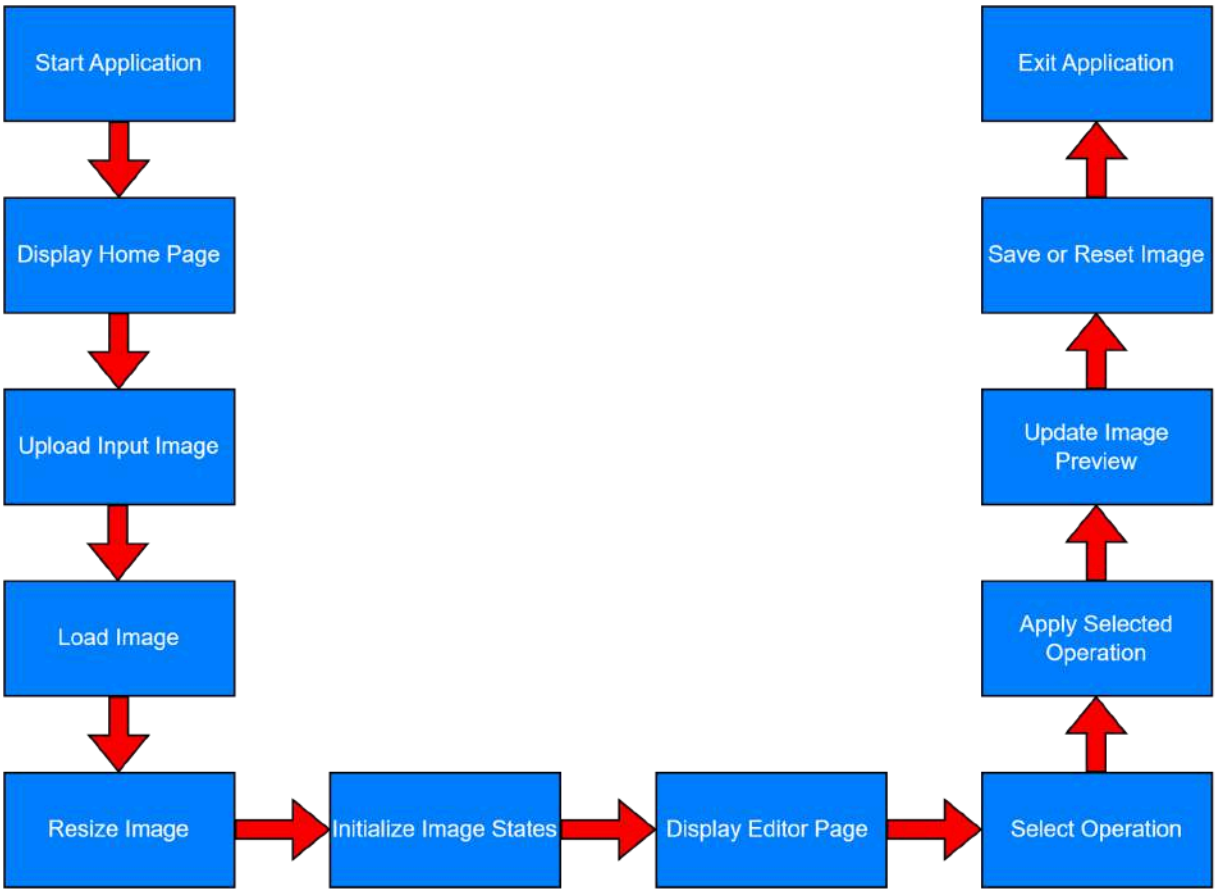
The user chooses the save option and selects the file location in the computer folder.

Output:

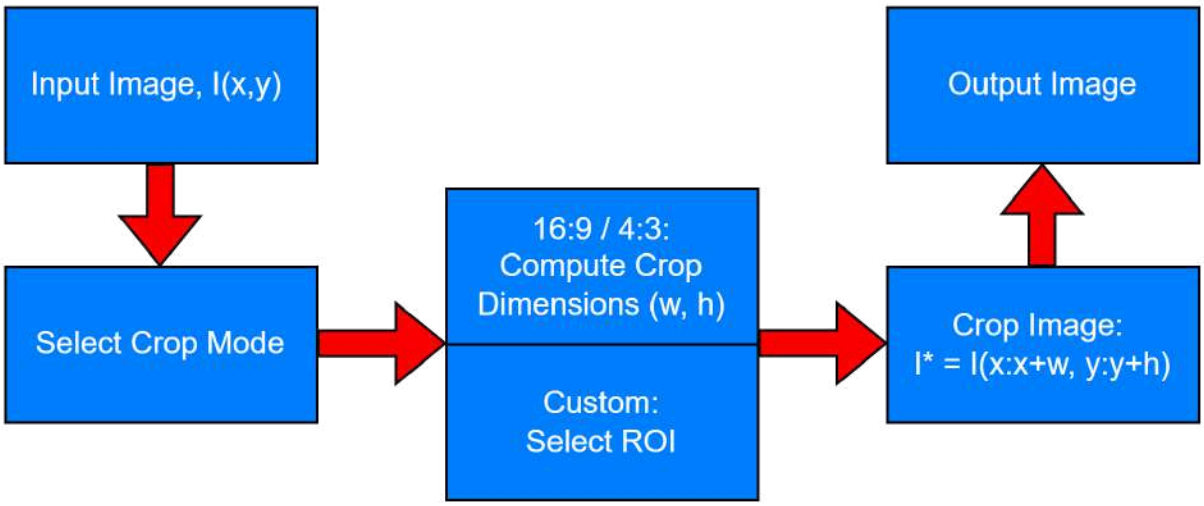
An edited image is saved in the specified location that has been chosen.

3.0 Process Flow Diagram

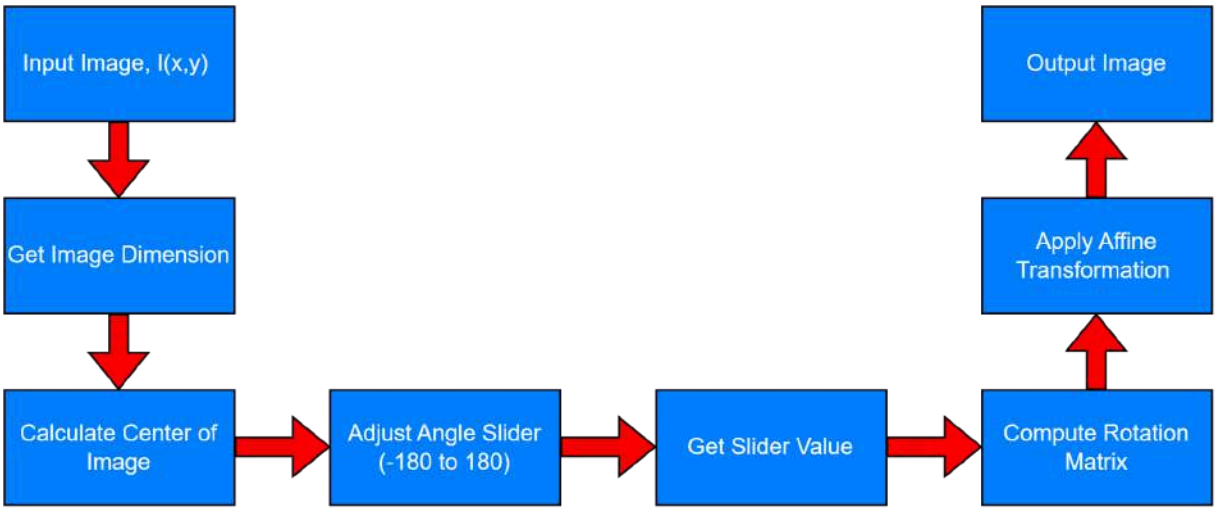
3.1 Overall Application Process Flow



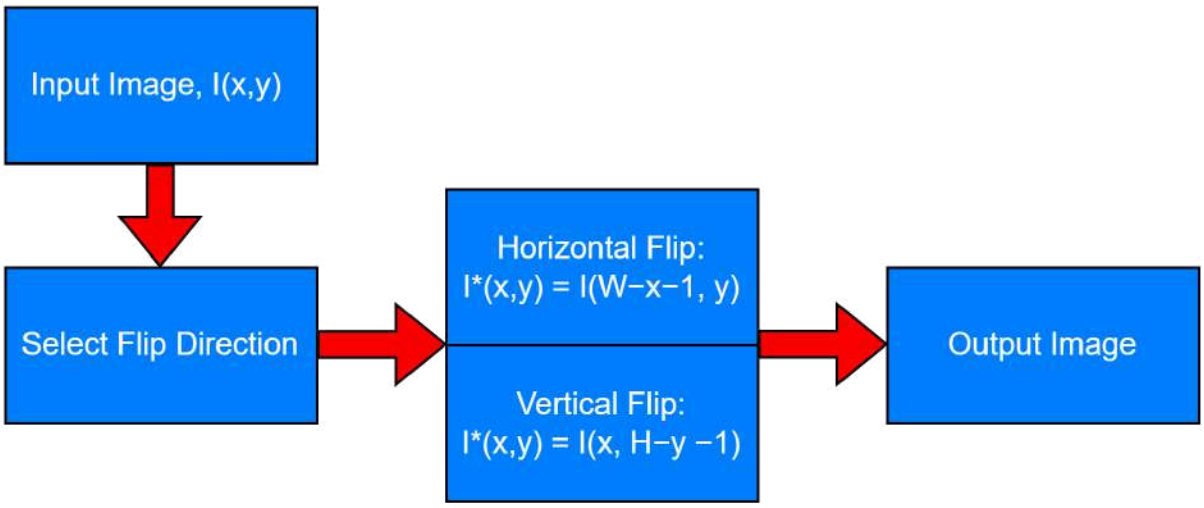
3.2 Image Cropping Process



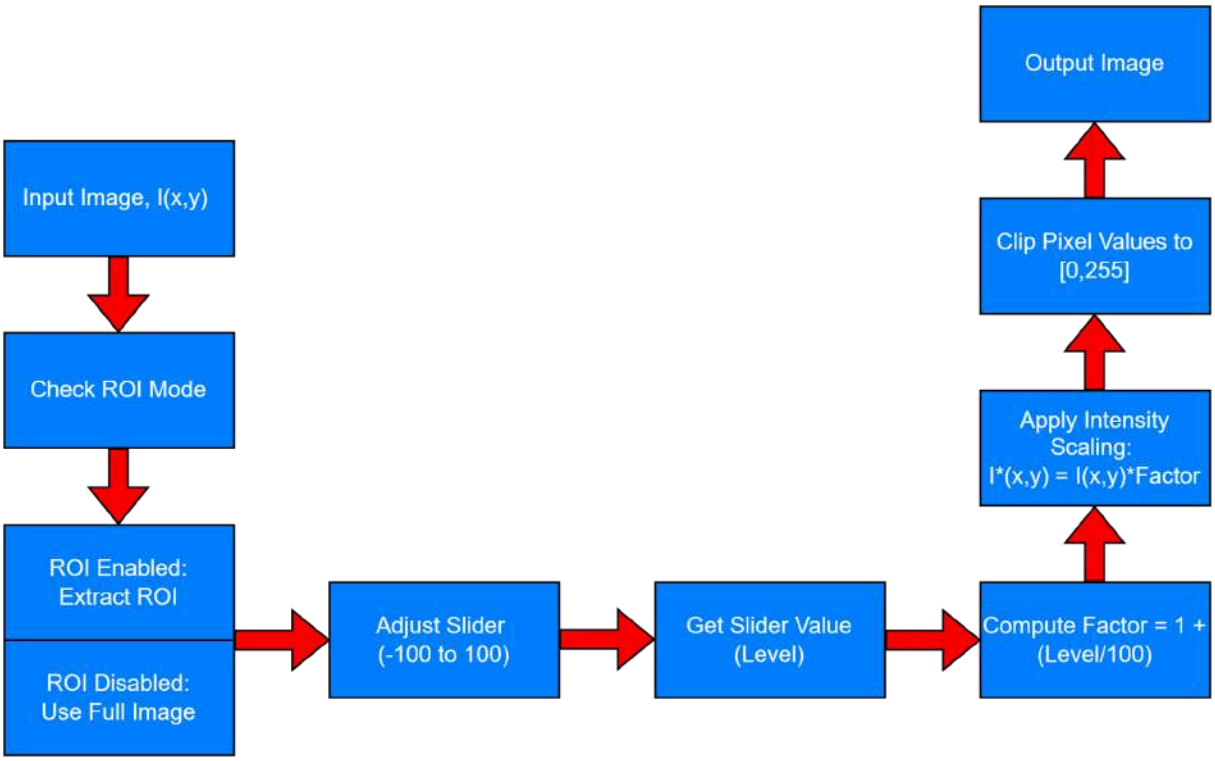
3.3 Image Rotation Process



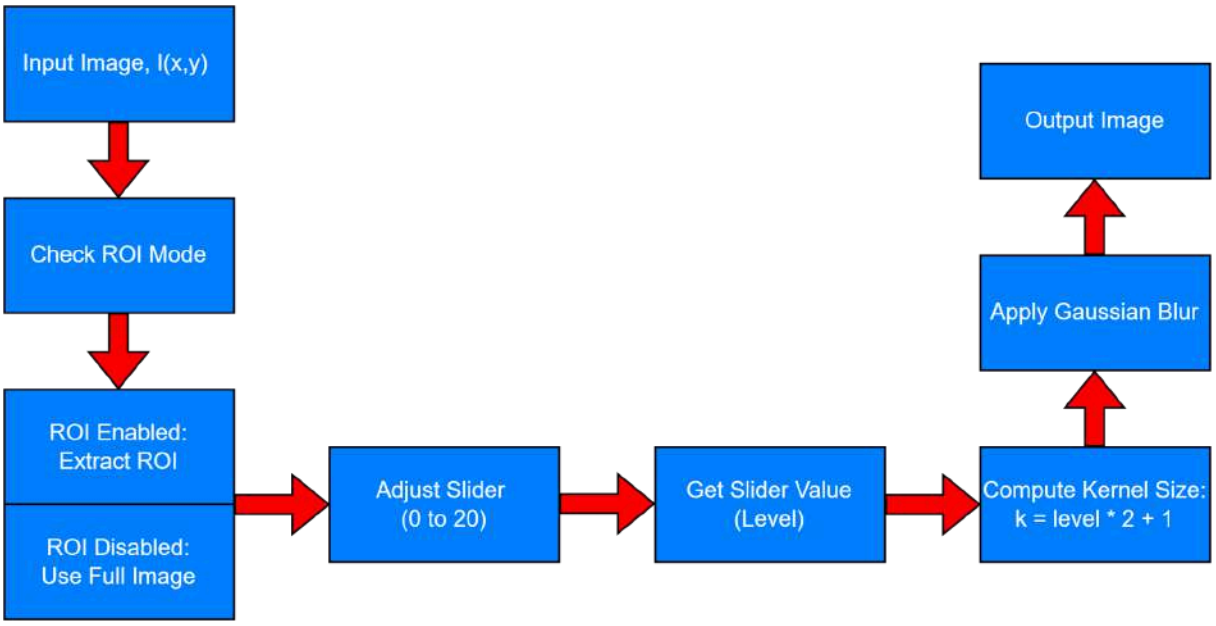
3.4 Image Flipping Process



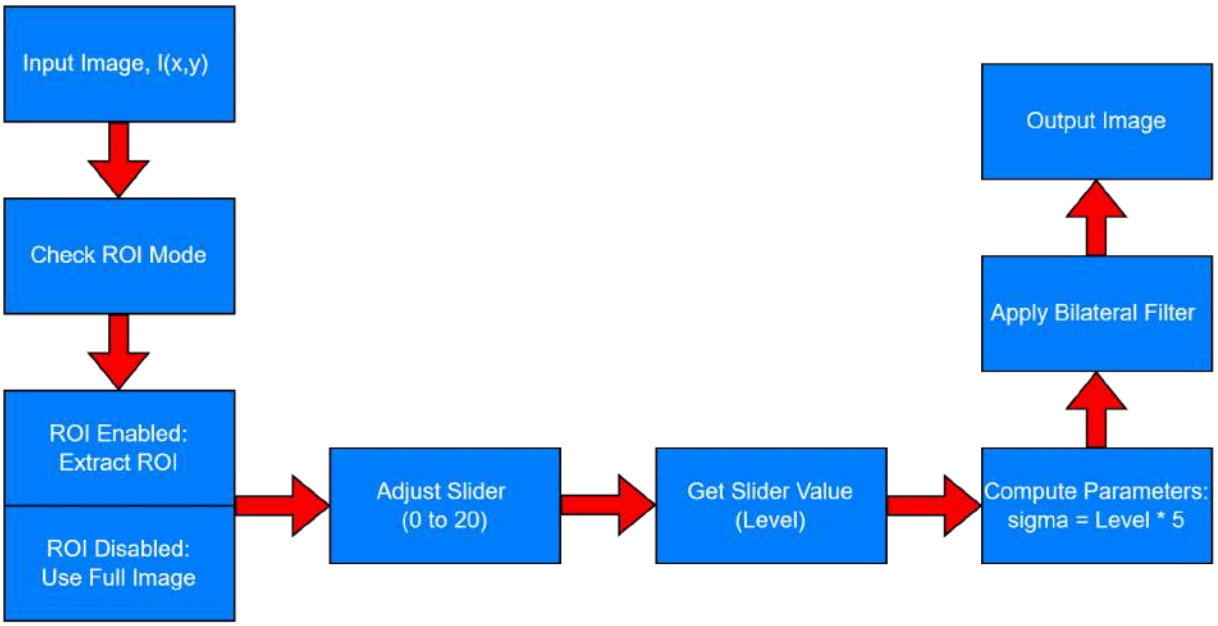
3.5 Brightness Adjustment Process



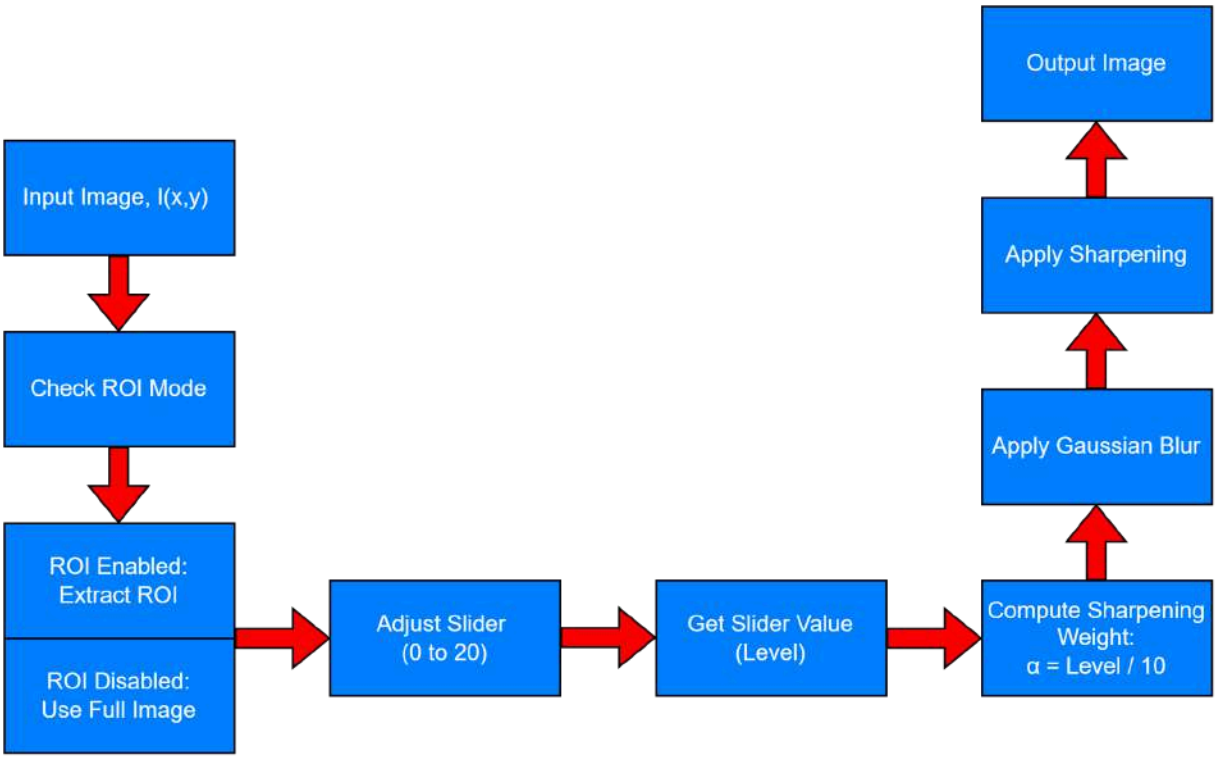
3.6 Image Blurring Process



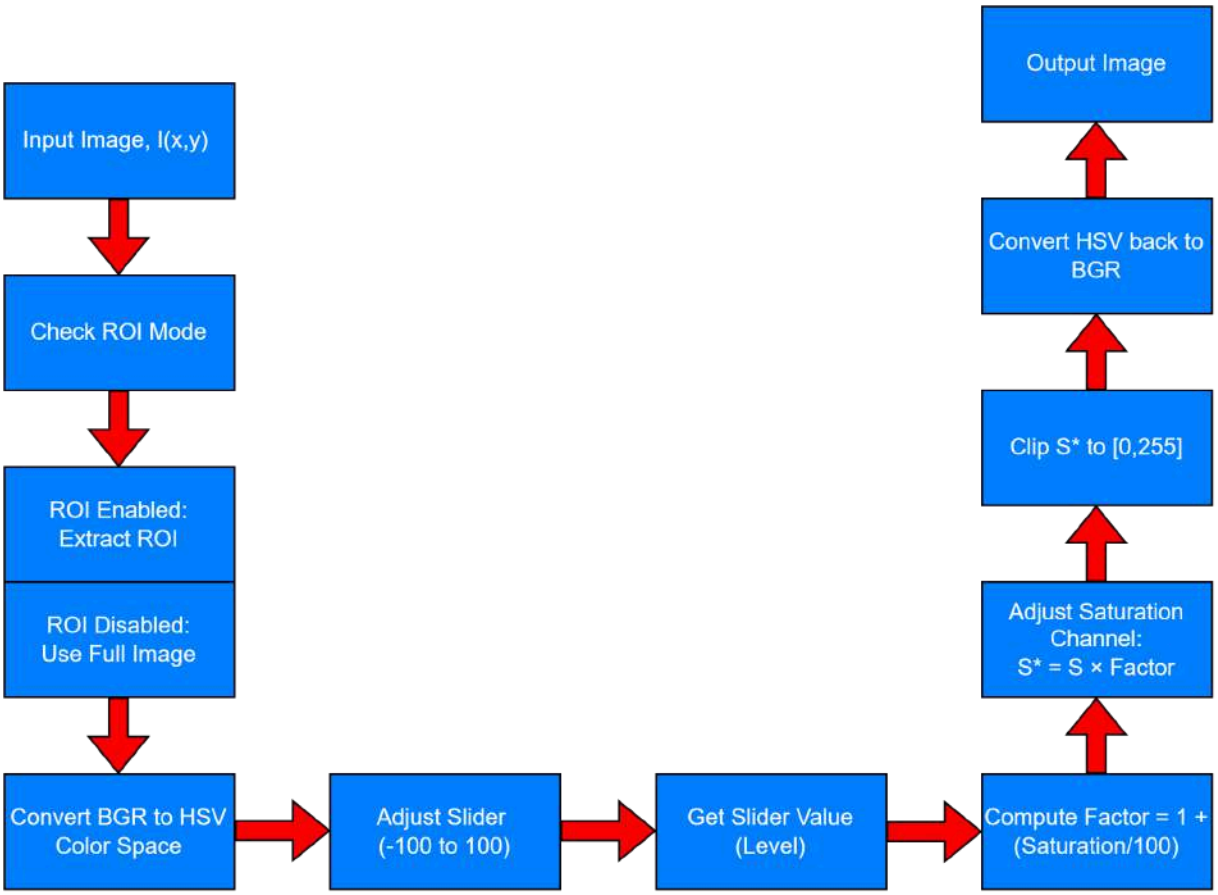
3.7 Noise Reduction Process



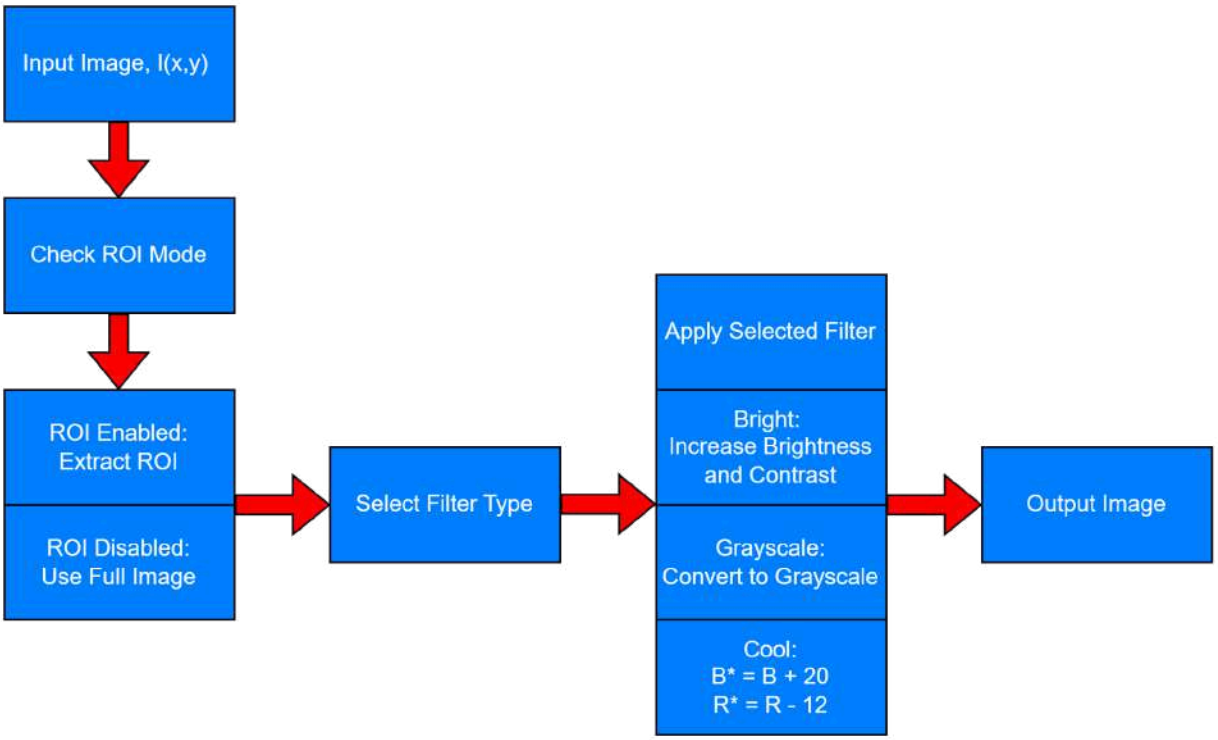
3.8 Image Sharpening Process



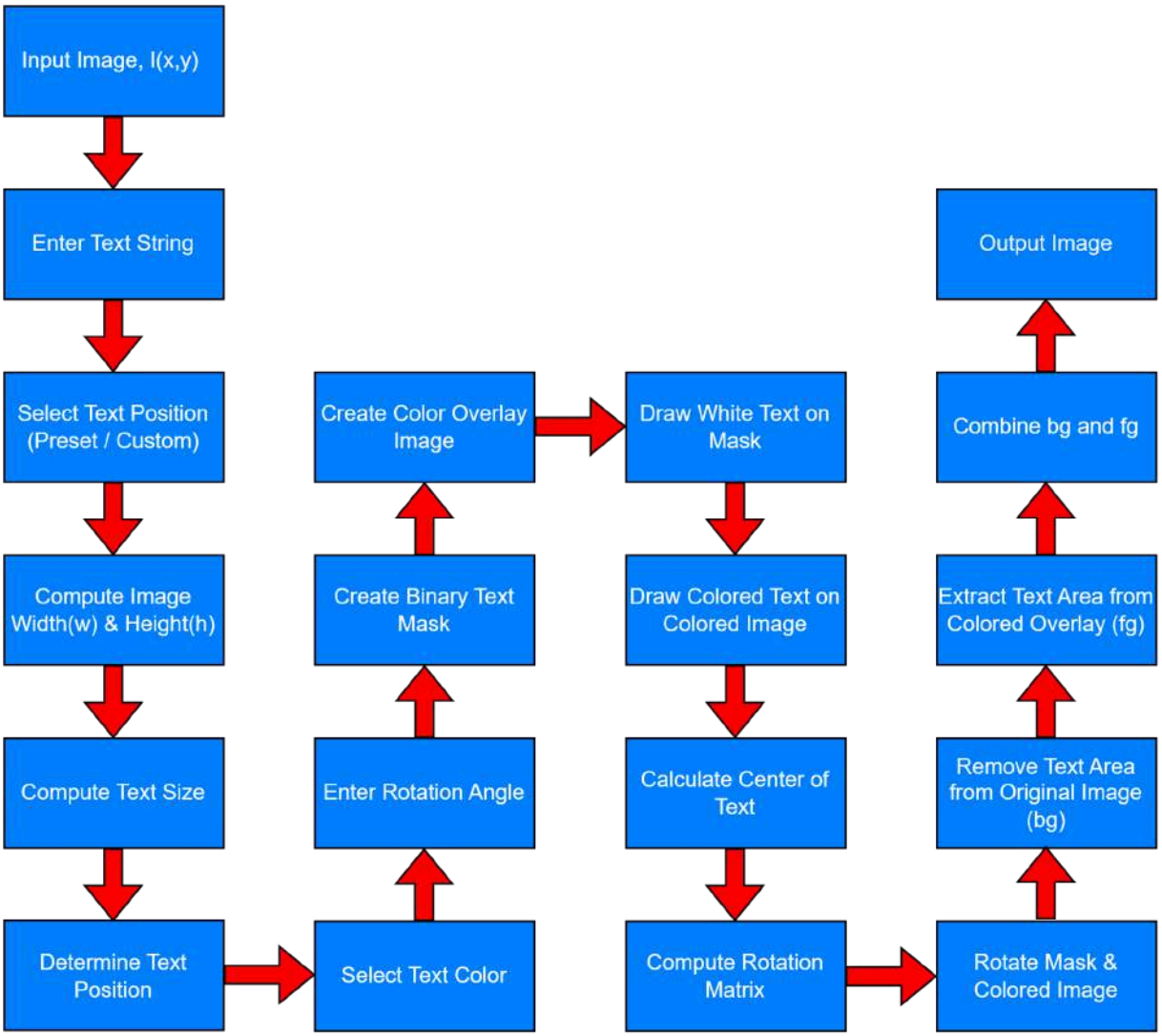
3.9 Saturation Adjustment Process



3.10 Visual Filtering Process



3.11 Text Insertion Process



4.0 Explanation of Process Flow

4.1 Overall Application Process Flow

From the overall application process flow diagram in 3.1, it displays and describes the general workflow of our group application, iPixelo beginning from the moment the user starts the program until the application is closed or exited. The program of iPixelo starts when the user launches or initiates the application. As the application starts, the system will display the home page interface where some basic information about the application is provided and the user is allowed to upload any desired input image file from his or her device. If the user does not feel like uploading his or her image or has no image to be uploaded, the user can choose any sample image provided by the system at the section below the upload section. Once the image is uploaded and chosen by the user, the system will load the image into memory. The image is then resized automatically in order to make sure that it fits properly within the application window while maintaining its original aspect ratio. After the image resizing process, the system of iPixelo initializes the image states by storing the original image as base image and preparing variables that will track and associate with all the editing operations. This process is done to ensure that all the future edits or operations are applied consistently using a processing pipeline. Next, the editor page is displayed. This page contains all the available editing tools and the image preview area where the user can witness the live changes applied to the image. Here, the user can choose or select any operation available such as crop, rotate, flip, brightness adjustment, image blurring, noise reduction, image sharpening, saturation adjustment, filtering and adding text. After an operation is selected, the iPixelo system will apply the chosen operation to the image based on the user's input parameters. The image processing function or operation is executed and the result is generated through the processing pipeline. The processed image is then displayed in the image preview area so that the user can witness the effect of the selected operation in real time. After that, the user can either repeat the same operation with different parameters and inputs or select other operations to further edit the image. If the user is not satisfied with the result, the user can choose to reset the image back to its original version by clicking the reset button. As the image is reverted back to its original version, the user can continue to select desired operations and apply them onto the image. Once the user is satisfied with the result, the user can save the image by clicking the save button. The user can decide the output image filename and its stored

location and then save the image. Finally, the program is terminated when the user chooses to exit the application by clicking onto the cross or x on the application window.

4.2 Image Cropping Process

From the image cropping process diagram in 3.2, it displays and describes the image cropping process that allows the user to remove unwanted regions of the image and adjust the image composition based on a selected cropping mode. The process of image cropping starts with an input image represented as I(x,y), where x and y indicate the pixel coordinates. After that, the user can select a crop mode from four available options which are Original, 16:9, 4:3 and Custom. If the user selects Original, there will be no cropping and the original image is returned directly as the output image. If the user selects 16:9 or 4:3, the system will compute a new crop dimension based on the selected aspect ratio. Let the original image size be w x h where w is width and h is height of the image. For 16:9 mode, the target aspect ratio is 16/9 while for 4:3 mode , the target aspect ratio is 4/3. The system will compare the current image ratio w/h with the target aspect ratio. If the image ratio is greater than the target ratio, the width is reduced while keeping the height constant using the formula new_w = h*aspect. If the image ratio is smaller than the target ratio, the height is reduced while keeping the width constant using the formula new_h = w/aspect. After that, the center of the cropping region is calculated by using formula x = (w-new_w)/2 and y = (h-new_h)/2. The cropped image is obtained and returned as I* = I(x:x+new_w, y:y+new_h). If the user selects Custom, the system will allow the user to manually select a Region of Interest (ROI) using the mouse through another pop up window. The ROI is defined by four parameters x which is the horizontal starting coordinate, y which is the vertical starting coordinate, cw which is the width of selected region and ch which is the height of the selected region. The cropped image is then computed using I* = I(x:x+cw, y:y+ch). After the cropping operation is completed by using any of the cropping modes, the system will output the cropped image and display it to the user at the image preview area as the final result of the cropping process.

4.3 Image Rotation Process

From the image rotation process diagram in 3.3, it displays and describes the image rotation process that allows the user to change the orientation of an image by adjusting the slider to change the angle value. The image rotation process starts with an input image represented as I(x,y) where x and y indicate the pixel coordinates. After that, the system will obtain the image dimensions where w is the width of the image and h is the height of the image. These values are used to calculate the center of the image for rotation to take place later using the formula, center = (w//2, h//2). This ensures that the image rotates around its center. Next the user adjust the angle of rotation using a slider which ranges from -180 degree to +180 degrees. A negative value on the slider indicates that the image rotates in clockwise direction while a positive value on the slider indicates that the image rotates in the anticlockwise direction. A value of 0 indicates no rotation is applied to the image. The system will then get the slider value and the selected angle from the slider will be stored as the rotation parameter, angle. After that, the iPixelo system will compute the 2D rotation matrix using the formula M=getRotationMatrix2D(center, angle, 1.0) where c is the center of rotation, angle is the rotation angle in degrees and 1.0 is the scaling factor. After computing the rotation matrix, the system will then apply the affine transformation to the image using the formula I* = cv.warpAffine(img, M, (w, h)). This operation rotates every pixel in the image based on the rotation matrix while keeping the output image size the same as the original image or previous image size before any rotation is applied. Finally the rotated image I* is generated and displayed as the output image of the rotation process in the image preview area.

4.4 Image Flipping Process

From the image flipping process diagram in 3.4, it displays and describes the image flipping process that creates a mirror version of the input image by reversing pixel position along a selected axis based on the user's selection. The image flipping process starts with an input image represented as I(x,y) where x and y indicate the pixel coordinates. After that, the user selects the flip direction. There are two flip directions provided which are the horizontal flip and vertical flip. The selected direction is stored as the flip mode parameter in mode. If mode is equal to "horizontal", the image is mirrored from left to right. Each pixel at position (x,y) in the original image is moved to a new position using the formula $I^*(x,y) = I(W-x-1, y)$ where W is the width of the image, x is the original horizontal coordinate and y is the original vertical coordinate. The value W-x-1 reflects the pixel across the vertical center line. If the mode is equal to "vertical", the image is mirrored from top to the bottom. Each pixel at position (x,y) is moved to a new position using the formula $I^*(x,y) = I(x, H-y-1)$ where H is the height of the image, x is the original horizontal coordinate and y is the original vertical coordinate. The value H-y-1 reflects the pixel across the horizontal center line. In this implementation, these operations are performed using a built-in flipping function, cv.flip(image, mode) where if the mode is equal to 1, it represents horizontal flipping while if the mode is equal to 0, it represents vertical flipping. After the selected flipping operation is applied, the resulting image is generated and displayed as the output image of the flipping process at the image preview area.

4.5 Brightness Adjustment Process

From the brightness adjustment process diagram in 3.5, it displays and describes the brightness adjustment process that modifies the brightness and darkness of an image by scaling pixel intensity values through user adjustment on the provided slider. The brightness adjustment process starts with an input image represented as I(x,y) where x and y indicate the pixel coordinates. The user is allowed to either perform this adjustment operation onto a selected region only through Region of Interest (ROI) selection or entire image. After that, the iPixelo system will check whether the ROI mode is enabled. If ROI mode is enabled, only the selected image will be processed in this process. If the ROI mode is disabled, the full image is used. After deciding that, the user adjusts the brightness slider which ranges from -100 to +100. A negative value of the brightness slider indicates the lowering of image brightness while a positive value of the brightness slider indicates the increasing of image brightness. A value of 0 indicates that the brightness of the image remains unchanged. The system will get the selected slider value and the selected slider is stored as the brightness level, Level. The system then computes a brightness scaling factor using the formula Factor = 1 + (Level/100). After computing the factor, the brightness adjustment is applied through intensity scaling using the formula I*(x, y) = I(x,y)*Factor. If ROI mode is enabled, this operation only applies to the pixels inside the ROI where x and y are the top-left coordinate of ROI and w and h is the width and height of ROI. After scaling, the pixel values may exceed the valid range of 0 to 255. Therefore, all the pixel values are clipped to the the range using function np.clip(I*(x,y), 0, 255) where I*(x,y) is the region or entire image that already applied intensity scaling. This ensures that the output remains a valid 8-bit image. Finally, the processed image is generated and displayed as the output image of the brightness adjustment process at the image preview area.

4.6 Image Blurring Process

From the image blurring process diagram in 3.6, it displays and describes the image blurring process that reduces the image detail by smoothing the pixel variation. The image blurring process starts with an input image represented as I(x,y) where x and y indicate the pixel coordinates. The user is allowed to either perform this adjustment operation onto a selected region only through Region of Interest (ROI) selection or entire image. After that, the iPixelo system will check whether the ROI mode is enabled. If ROI mode is enabled, only the selected image will be processed in this process. If the ROI mode is disabled, the full image is used. After deciding that, the user adjusts the blur slider which ranges from 0 to 20. A larger value indicates the increase in the blurring strength of the image while a value of 0 indicates that there is no blur applied to the image. The system then will get the selected slider value and store it as the blur level, Level. After that, the system will computes the gaussian blue kernel size using formula k = Level*2+1. This ensures that the kernel size is always an odd number which is required for Gaussian filtering. After getting kernel size, the system will apply gaussian blur filtering to the image using the function cv.GaussinaBlur(image, k, 0). If ROI mode is enabled, this operation only applies to the pixels inside the ROI where x and y are the top-left coordinate of ROI and w and h is the width and height of ROI. Pixels outside the ROI remain unchanged. Finally, the blurred image is processed and displayed as the output image of the image blurring process at the image preview area.

4.7 Noise Reduction Process

From the Noise Reduction process diagram in 3.7, it displays and describes the noise reduction process that aims to remove unwanted noise from an image while preserving important edges. The noise reduction process starts with an input image represented as I(x,y) where x and y indicate the pixel coordinates. The user is allowed to either perform this adjustment operation onto a selected region only through Region of Interest (ROI) selection or entire image. After that, the iPixelo system will check whether the ROI mode is enabled. If ROI mode is enabled, only the selected image will be processed in this process. If the ROI mode is disabled, the full image is used. After deciding that, the user adjusts the noise reduction slider which ranges from 0 to 20. A larger value indicates a stronger noise reduction of the image while a value of 0 indicates that there is no noise reduction applied to the image. After that, the system will get the selected slider value and store it as the noise reduction level, Level. The system will then compute the filter parameter for the bilateral filter. The diameter of the pixel neighborhood is fixed at d = 9 while the standard deviation values for the color and space are computed as sigma = Level*5. After computing the filter parameter, the bilateral filter is then applied to the image using the function cv.bilateralFilter(image, d, sigma, sigma). If ROI mode is enabled, this operation only applies to the pixels inside the ROI where x and y are the top-left coordinate of ROI and w and h is the width and height of ROI. Pixels outside the ROI remain unchanged. Finally, the noise-reduced image is processed and displayed as the output image of the noise reduction process at the image preview area.

4.8 Image Sharpening Process

From the Image Sharpening process diagram in 3.8, it displays and describes the image sharpening process that enhances edges and fine details by emphasizing differences between pixels. The image sharpening process starts with an input image represented as I(x,y) where x and y indicate the pixel coordinates. The user is allowed to either perform this adjustment operation onto a selected region only through Region of Interest (ROI) selection or entire image. After that, the iPixelo system will check whether the ROI mode is enabled. If ROI mode is enabled, only the selected image will be processed in this process. If the ROI mode is disabled, the full image is used. After deciding that, the user adjusts the sharpening slider which ranges from 0 to 20. A larger value indicates an increase in the sharpening strength of the image while a value of 0 indicates that there is no sharpening applied to the image. After that, the system will get the selected slider value and store it as the sharpening level, Level. The system will then compute the sharpening weight using formula α= Level/10. After computing the sharpening weight, α, the system will now apply gaussian blur to create a smoothed version of the image first using B = cv.GaussinaBlur(image, (3,3), 1.0) where (3,3) indicates 3x3 kernel size and 1.0 indicates the scaling factor. After that, the system will apply the sharpening onto the smooth version of image using weighted image combination function cv.addWeighted(image, 1.0 + α, B, -α, 0) where α is the sharpening weight and B is the image that has been applied to gaussian filtering. This operation increases the difference between the original image and its blurred version, making the edges more visible. If ROI mode is enabled, this operation only applies to the pixels inside the ROI where x and y are the top-left coordinate of ROI and w and h is the width and height of ROI. Pixels outside the ROI remain unchanged. Finally, the sharpened image is processed and displayed as the output image of the sharpening process at the image preview area.

4.9 Saturation Adjustment Process

From the Saturation Adjustment process diagram in 3.8, it displays and describes the saturation adjustment process that modifies the intensity of colors in an image. The saturation adjustment process starts with an input image represented as I(x,y) where x and y indicate the pixel coordinates. The user is allowed to either perform this adjustment operation onto a selected region only through Region of Interest (ROI) selection or entire image. After that, the iPixelo system will check whether the ROI mode is enabled. If ROI mode is enabled, only the selected image will be processed in this process. If the ROI mode is disabled, the full image is used. Next, the system will convert the image from BGR color space to the HSV color space using the function cv.cvtColor(image, cv.COLOR_BGR2HSV).astype(np.float32). After that, the user adjusts the saturation slider which ranges from -100 to 100. A negative value indicates the reduction in the saturation of the image while a positive value indicates the increase in the saturation of the image. A value of 0 indicates that the image remains unchanged. After that, the system will get the selected slider value and store it as the Saturation level, Level. The system will then compute the scaling factor using Factor = 1 + Level/100. After getting the scaling factor, the saturation channel is adjusted and clipped using the formula or function np.clip(hsv*Factor, 0, 255). The modified HSV image is then converted back to BGR color space using the function cv.cvtColor(hsv.astype(np.uint8), cv.COLOR_HSV2BGR). If ROI mode is enabled, this operation only applies to the pixels inside the ROI where x and y are the top-left coordinate of ROI and w and h is the width and height of ROI. Pixels outside the ROI remain unchanged. Finally, the resulting image is processed and displayed as the output image of the saturation adjustment process at the image preview area.

4.10 Visual Filtering Process

From the Visual Filtering process diagram in 3.10, it displays and describes the visual filtering process that applies predefined visual styles to an image to change its overall appearance. The visual filtering process starts with an input image represented as I(x,y) where x and y indicate the pixel coordinates. The user is allowed to either perform this adjustment operation onto a selected region only through Region of Interest (ROI) selection or entire image. After that, the iPixelo system will check whether the ROI mode is enabled. If ROI mode is enabled, only the selected image will be processed in this process. If the ROI mode is disabled, the full image is used. After that, the user selects a filter type from 4 available options which are original, bright filter, grayscale filter (B&W) and cool filter. If the original option is selected, no filter will be applied and the output image will remain the same. If the bright filter is selected, the brightness and contrast of the image is increased using the function cv.convertScaleAbs(image, alpha=1.15, beta=5) where alpha is the contrast scaling factor and beta is the brightness offset. If the grayscale filter is selected, the image will be converted to black and white using the function cv.cvtColor(image, cv.COLOR_BGR2GRAY) to change the BGR image to grayscale and cv.cvtColor(grayscale_image, cv.COLOR_GRAY2BGR) to convert back to BGR format for display. If the cool filter is selected, the image will be applied to a cooler tone. To achieve this, the system will increase the blue channel using the formula cool[:, :, 0] += 20 and increase the red channel using the formula cool[:, : , 2] -= 12 while the green channel remains unchanged. After that, all the channel values are clipped to the range 0 to 255. If ROI mode is enabled, this visual filter only applies to the pixels inside the ROI where x and y are the top-left coordinate of ROI and w and h is the width and height of ROI. Pixels outside the ROI remain unchanged. Finally, the filtered image is processed and displayed as the output image of the visual filtering process at the image preview area.

4.11 Text Insertion Process

From the Text Insertion process diagram in 3.11, it displays and describes the text insertion process that allows the user to add customized text onto the image with selected position, color and rotation. The text insertion process starts with an input image represented as I(x,y) where x and y indicate the pixel coordinates. After that, the user enters a text string. This text will be drawn onto the image using a fixed font style and size. In the system, the font used is Hershey Simplex, with font size=2 and thickness=4. Next, the user selects the text position either preset positions including top-left, top-center, top-right, mid-left, center, mid-right, bottom-left, bottom-center and bottom-right or custom position where user manually enters the horizontal coordinate x and the vertical coordinate y. To compute preset positions, the system will first get the image width, W and height, H and computes the text size by getting the text width, tw and height, th using the function cv.getTextSize(text, cv.FONT_HERSHEY_SIMPLEX,2,4). After that the final text position is determined based on these values. Next, the user selects the text color where the available colors include black, white, red, yellow, orange and blue and all of them are in BGR format. Then the user enters a rotation angle using a spin box with range -360 to +360 degrees where negative values indicate that the text rotates in clockwise direction while positive values indicate that the text rotates in anticlockwise rotation. To apply rotation, the system will then need to create a binary mask image where white text is drawn and then a colored overlay image where the colored text is drawn. After creating both of them, white text is drawn onto the binary mask and colored text is drawn onto the colored overlay image. Then the system will calculate the center of the text where cx = x+tw//2 and cy = y-th//2. Using this center and the rotation angle, a rotation matrix is computed, M = getRotationMatrix2D((cx,cy), angle, 1.0). After that, both the binary mask and colored overlay image are rotated using warpAffine. Next, the system removes the text area from the original image using the rotated mask, producing a background image. Then the colored rotated text is extracted as the foreground from the colored overlay image. Finally, the background and foreground are combined. This produces the final image with inserted text at the selected position, color, rotation and is then displayed as the output image of the text insertion process at the image preview area.

5.0 Results & Discussion

5.1 Crop Image

Sample No.	Input Image	Output Image
1		
2		

The above image cropping result demonstrates the effectiveness of the custom cropping mode by using a region of interest (ROI). In both samples, the cropping operation successfully removes unnecessary background areas while preserving the main subject of interest in the image. Sample 1 focuses on the architectural structure by eliminating peripheral regions while Sample 2 isolates the primary subject from the surrounding environment. Therefore, this feature provides users a flexible and precise control over image composition, allowing them to highlight important regions and improve visual clarity by removing unwanted areas from the image.

5.2 Rotate Image

Sample No.	Input Image	Output Image
1		
2		

From the above rotation result, it demonstrates the effectiveness of image rotation operation when a specified rotation angle is applied. In both samples, the input images are rotated accurately and change their orientation while preserving the visual content and structural integrity of the images. Sample 1 shows that the architectural elements are correctly rotated without distortion, while Sample 2 shows that the subject remains clearly visible after rotation. Therefore, this feature enables users to correct image orientation and adjust visual alignment easily in order to ensure that images are displayed in the desired orientation.

5.3 Flip Image

Sample No.	Input Image	Output Image
1		
2		

The above image flipping result demonstrates how the flip operation changes the orientation of the image by mirroring it along a selected axis. In both samples, a horizontal flip was applied. For sample 1, the original image of the twin tower is flipped horizontally, causing the left and right sides of the image to be reversed. We can observe that the position of buildings, trees and background elements appears mirrored but the overall structure and visual quality of the image remain unchanged. This shows that the flipping does not alter pixel values but only their positions. For sample 2, it is the same, the image of the motorcyclist is flipped horizontally without altering the overall structured and visual quality. Therefore, this feature allows users to easily change image orientation through mirroring for correcting composition.

5.4 Adjust Image Brightness

Sample No.	Input Image	Output Image
1		
2		

The brightness adjustment results show how changing the brightness level affects the overall brightness and darkness of the image. For both sample 1 and sample 2, the brightness level is reduced, making the output images appear to be dimmer and darker than the original images. However, we still can see the objects inside the image even though the images are darkened. Overall, the brightness adjustment function works effectively by scaling pixel intensity values, allowing users to adjust images that are too bright or too dark. However, extreme darkening and brightening will cause loss of detail in certain areas of the image causing the image to lose its quality, thus moderate adjustment values will give the best visual result. Therefore, this feature allows users to control image brightness effectively for visibility enhancement.

5.5 Adjust Image Blur

Sample No.	Input Image	Output Image
1		
2		

The blur adjustment results show how increasing the blur level reduces the image sharpness and fine details. For both sample 1 and sample 2, a Gaussian blur with high blur level is applied to the images, resulting in the output image appearing significantly softer and fine details on the objects inside the images are no longer clearly visible. The blur function works by applying a Gaussian filter with a kernel size based on the slider value. Higher blur levels produce smoothing effects however reducing the visual details. Therefore, this feature enables users to create soft-focus or background blurring effects by smoothing image details for emphasizing subjects.

5.6 Adjust Image Noise Reduction

Sample No.	Input Image	Output Image
1		
2		

The noise reduction results demonstrate how the bilateral filter smooths the images while preserving important edges. For sample 1, after noise reduction is applied, the output image appears slightly smoother, especially the sky and shadow areas where small grainy textures are reduced but at the same time the edges of the building remain clear. For sample 2, after applying noise reduction, the background and darker region under the bridge look smoother and less grainy while the rider and the motorcycle edges are still well defined. Therefore, the noise reduction feature allows users to improve image quality while maintaining essential visual details especially under low light or noisy conditions.

5.7 Adjust Image Sharpness

Sample No.	Input Image	Output Image
1		
2		

The sharpening results show how increasing the sharpness level enhances the edge details and makes the image appear to be clearer. In sample 1, the sharpening operation applied makes the building edges, window lines and structural details appear to be more distinct compared to the original image. In sample 2, the outlines of the rider, helmet and motorcycle become more defined and make the subject stand out more clearly from the background. Therefore, this sharpening feature enables users to strengthen image edges and enhance fine details for better visual clarity and making important subjects more distinguishable from the image.

5.8 Adjust Image Saturation

Sample No.	Input Image	Output Image
1		
2		

The saturation adjustment results show how changing the saturation level affects the intensity of colors in an image. For sample 1, we can clearly observe that the sky appears to be bluer and the buildings show stronger color tones as the saturation level is increased. For sample 2, we can observe that the color of the motorcycle, rider's cloth and background become more intense, making the subject stand out more clearly from the surroundings after the saturation level is increased. Therefore, the saturation adjustment feature effectively enhances the color richness, making the image to be more visually attractive for the users.

5.9 Apply Image Filter

Sample No.	Input Image	Output Image
1		
2		

The filter results demonstrate how predefined visual styles change the overall appearance of an image. For sample 1, the cool filter is applied to the image. The output image appears to be slightly bluish and the warm tones are reduced. This creates a cooler and calmer visual mood compared to the original image while the main structure and details of the image remain clearly visible. Same for sample 2, the cool filter is applied and the output image shows stronger blue tones. Therefore, this feature allows the users to change the overall mood or visual style of the image by applying the predefined filters without manually adjusting multiple image parameters.

5.10 Insert Text for Image

Sample No.	Input Image	Output Image
1		
2		

From the above result, it demonstrates that the text insertion feature functions correctly by allowing the text to be overlaid onto the input image at user-selected positions and rotation angle. In both samples, the added text, “I Love KL” is clearly visible and does not hide the main subject of the image. The text is rendered consistently with the chosen color, position and rotation indicating that the text insertion operation performs as intended. Therefore, this feature provides a simple and effective way for users to add meaningful textual information or captions to images.

5.11 ROI Based Adjustment

Sample No.	Input Image	Output Image
1		
2		

The results above demonstrate the effectiveness of ROI-based adjustment, where image enhancement operations are applied selectively to a user selected ROI while the remaining areas of the image remain unchanged. In both samples, adjustments such as brightness, blurring, noise reduction, sharpening, and saturation are confined to the selected regions which clearly highlight the targeted effects. This selective processing enables localized enhancement without affecting the overall image composition. Therefore, this ROI-based adjustment feature provides precise control over specific regions of an image for users to apply targeted enhancements efficiently while preserving the visual integrity of surrounding areas in the image.

6.0 Workload Distribution

Members	Tasks	Task Percentage
Chuah Chun Yi	System Design (20%), Solution Contribution (60%), Report writing (20%)	34%
Chong Jun Hong	System Design (20%), Solution Contribution (20%), Report writing (60%)	33%
Tai Yi Tian	System Design (60%), Solution Contribution (20%), Report writing (20%)	33%

7.0 Conclusion

In conclusion, the development of iPixelo has successfully achieved the project's objective which is to design and implement a functional and high-fidelity image editing application that integrates fundamental image processing concepts. By utilizing Python, the OpenCV library and the PyQt6-based GUI, this project successfully demonstrates the practical application of image processing techniques through an interactive and user-oriented system.

The implemented application, iPixelo fulfills all the requirements of the project guideline. It provides a user-friendly interface that allows users to upload an input image, select various editing and enhancement operations and save the processed output image to local storage. Furthermore, the interactive parameter controls such as sliders and selection buttons allows users to adjust processing parameters including brightness, blurring, sharpening, noise reduction and saturation in real time. The real-time preview functionality allows users to easily get the immediate effects of each operation before saving it as a file in local storage.

Technically, this project successfully implemented a lot of image processing operations including geometric transformations such as rotation, flipping and cropping as well as image enhancement and filtering techniques such as noise reduction, sharpening and intensity adjustment. iPixelo further supports ROI-based processing allowing users to apply specific operations to selected areas of an image. In addition, this application applies a non-destructive editing workflow which ensures flexibility and preserving image integrity during editing.

Overall, iPixelo is an interactive image editing application based on the theoretical knowledge and practical skills learned throughout the semester. This project has demonstrated a clear understanding of fundamental image processing concepts and their real-world implementation, which achieve the goal of integrating image processing techniques into a functional application.